This Java 8 challenge tests your knowledge of Lambda expressions!

Write the following methods that return a lambda expression performing a specified action:

1. PerformOperation isOdd(): The lambda expression must return *true* if a number is odd or *false* if it is even.
2. PerformOperation isPrime(): The lambda expression must return *true* if a number is prime or *false* if it is composite.
3. PerformOperation isPalindrome(): The lambda expression must return *true* if a number is a palindrome or *false* if it is not.

Input Format

Input is handled for you by the locked stub code in your editor.

Output Format

The locked stub code in your editor will print T lines of output.

Sample Input

The first line contains an integer, T (the number of test cases).

The T subsequent lines each describe a test case in the form of 2 space-separated integers:

The first integer specifies the condition to check for (1 for Odd/Even, 2 for Prime, or 3 for Palindrome). The second integer denotes the number to be checked.

```
5
```

```
        if (a < 2)
            return false;
        for (int i = 2; i * i <= a; i++) {
            if (a % i == 0)
                return false;
        }
        return true;
    }
};
```

Line: 69 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Your Output (stdout)

```
EVEN
PRIME
PALINDROME
ODD
COMPOSITE
```

Problem

Submissions

Leaderboard

Discussions

Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array.

The Player class is provided for you in your editor. It has 2 fields: a $name$ String and a $score$ integer.

Given an array of n Player objects, write a comparator that sorts them in order of decreasing score; if 2 or more players have the same score, sort those players alphabetically by name. To do this, you must create a Checker class that implements the Comparator interface, then write an int compare(Player a, Player b) method implementing the Comparator.compare(T o1, T o2) method.

Input Format

Input from stdin is handled by the locked stub code in the Solution class.

The first line contains an integer, n, denoting the number of players.
Each of the n subsequent lines contains a player's $name$ and $score$, respectively.

Constraints

- $0 \leq score \leq 1000$
- 2 players can have the same name.
- Player names consist of lowercase English letters.

Output Format

You are not responsible for printing any output to stdout. The locked stub code in Solution will create a Checker object, use it to sort the Player array, and print each sorted element.

Sample Input

```
5
amy 100
david 100
```

```
25
26          Player[] player = new Player[n];
27          Checker checker = new Checker();
28
29          for(int i = 0; i < n; i++){
30              player[i] = new Player(scan.next(), scan.nextInt());
31          }
32          scan.close();
33
34          Arrays.sort(player, checker);
35          for(int i = 0; i < player.length; i++){
36              System.out.printf("%s %s\n", player[i].name, player[i].score);
37          }
38      }
39  }
```

Line: 2 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin) Download

```
5
amy 100
david 100
heraldo 50
aakansha 75
```

Problem

Submissions

Leaderboard

You are given a list of student information: ID, FirstName, and CGPA. Your task is to rearrange them according to their CGPA in decreasing order. If two student have the same CGPA, then arrange them according to their first name in alphabetical order. If those two students also have the same first name, then order them according to their ID. No two students have the same ID.

Hint: You can use comparators to sort a list of objects. See the oracle docs to learn about comparators.

Input Format

The first line of input contains an integer N, representing the total number of students. The next N lines contains a list of student information in the following structure:

```
ID Name CGPA
```

Constraints

$2 \leq N \leq 1000$

$0 \leq ID \leq 100000$

$5 \leq |Name| \leq 30$

$0 \leq CGPA \leq 4.00$

The name contains only lowercase English letters. The ID contains only integer

Line: 72 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

You have earned 10.00 points!

You are now 30 points away from the 3rd star for your java badge.

0% 50/80

Congratulations

You solved this challenge. Would you like to challenge your friends?

Next Challenge

Test case 0

Test case 1

Compiler Message

Success

33°C Partly sunny

Search

ENG IN

11:46 11-08-2026

Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array. The Player class is provided in the editor below. It has two fields:

1. $name$: a string.
2. $score$: an integer.

Given an array of n Player objects, write a comparator that sorts them in order of decreasing score. If 2 or more players have the same score, sort those players alphabetically ascending by name. To do this, you must create a Checker class that implements the Comparator interface, then write an int compare(Player a, Player b) method implementing the Comparator.compare(T o1, T o2) method. In short, when sorting in ascending order, a comparator function returns -1 if $a < b$, 0 if $a = b$, and 1 if $a > b$.

Declare a Checker class that implements the comparator method as described. It should sort first descending by score, then ascending by name. The code stub reads the input, creates a list of Player objects, uses your method to sort the data, and prints it out properly.

Example

$n = 3$ $data = [[Smith, 20], [Jones, 15], [Jones, 20]]$

Sort the list as $data_{sorted} = [[Jones, 20], [Smith, 20], [Jones, 15]]$. Sort first descending by score, then ascending by name.

```
22  }
23
24 > ...
```

Line: 24 Col: 1

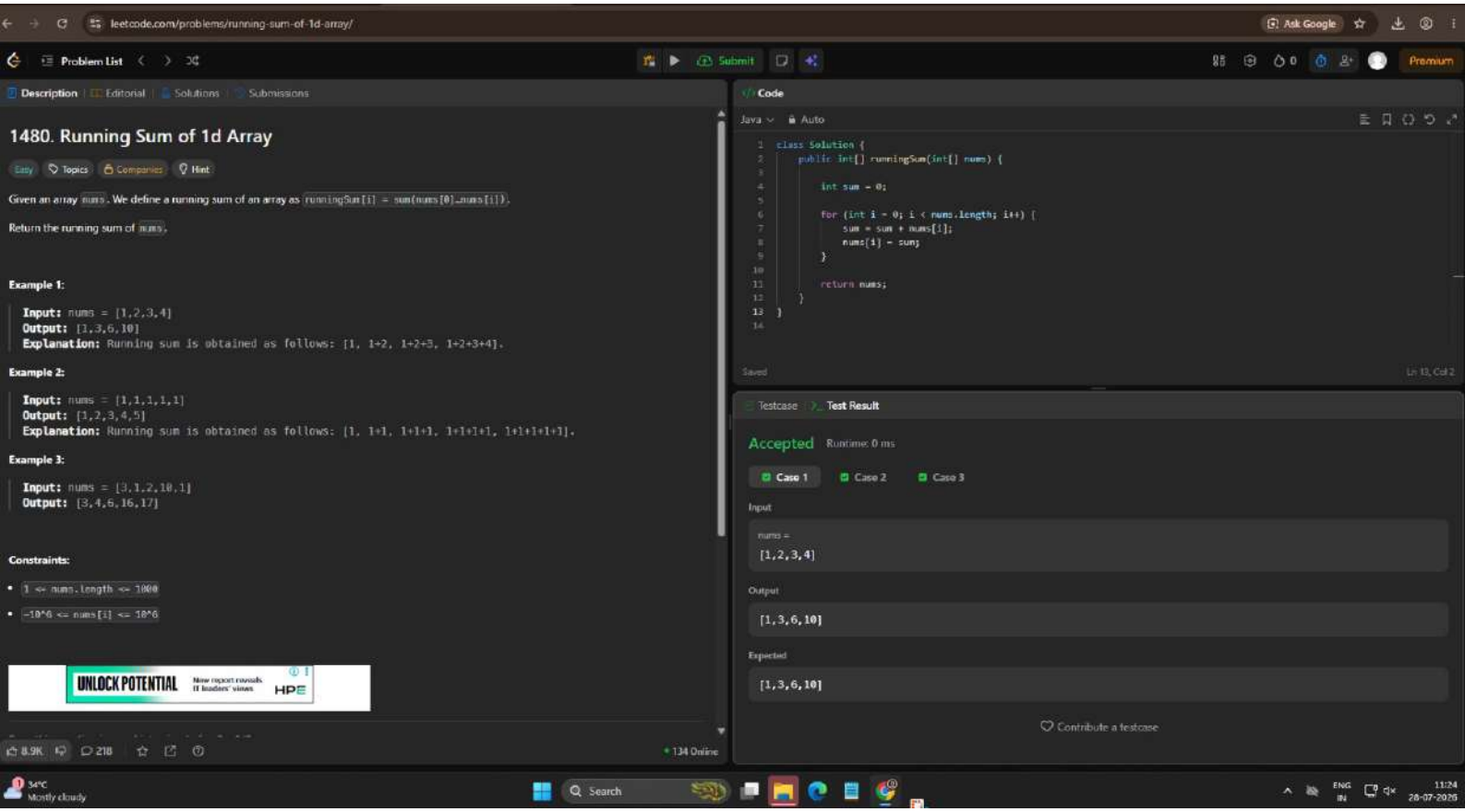
Upload Code as File | Test against custom input | Run Code | Submit Code

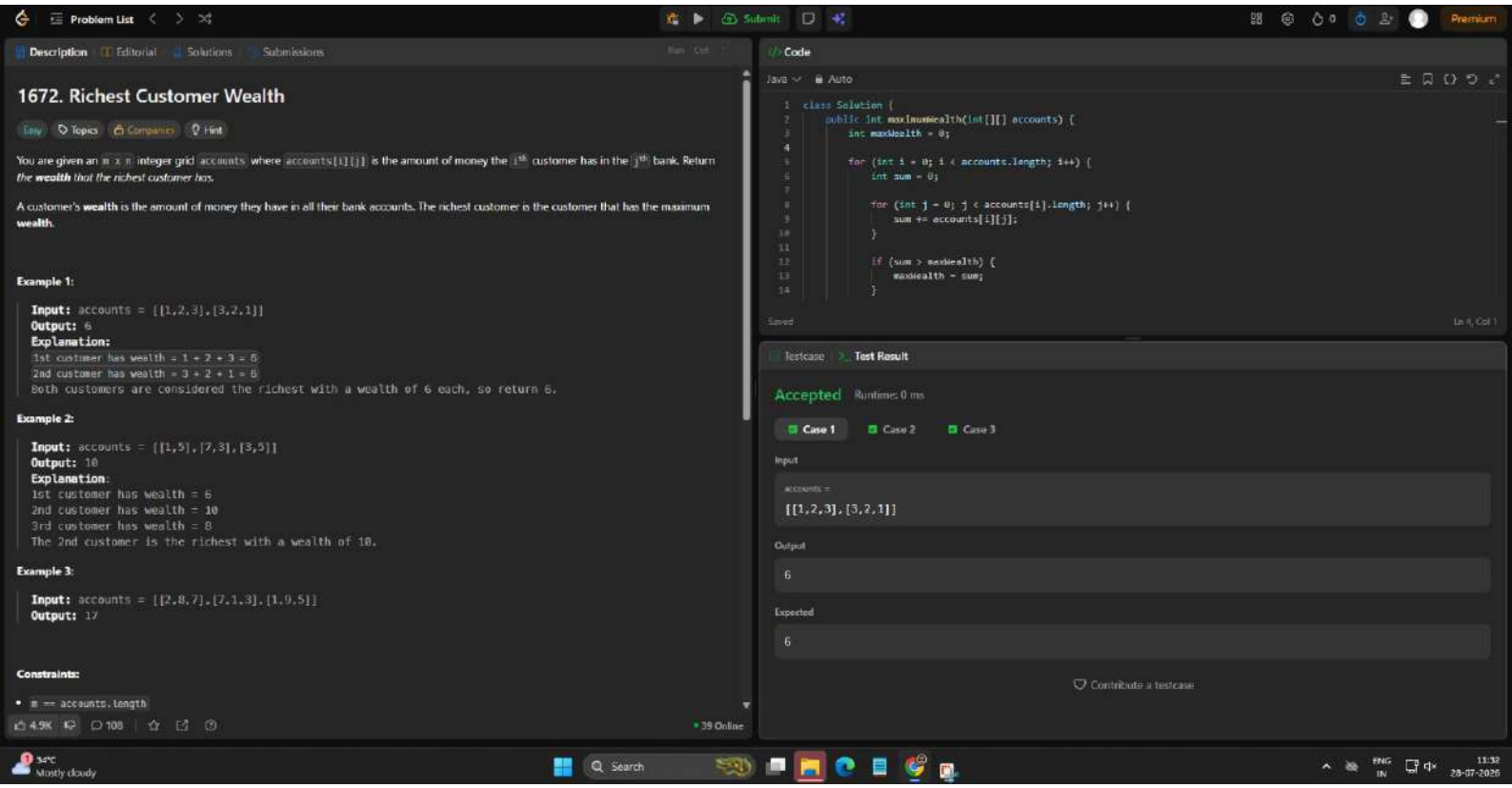
Congratulations

You solved this challenge. Would you like to challenge your friends?

Next Challenge

Test case 0 | Compiler Message


leetcode.com/problems/running-sum-of-1d-array/
Ask Google
Problem List
Submit
Premium
Description | Editorial | Solutions | Submissions
1480. Running Sum of 1d Array
Easy
Topics
Companies
Hint
Given an array nums. We define a running sum of an array as runningSum[i] = sum(nums[0]…nums[i]).
Return the running sum of nums.
Example 1:
Input: nums = [1,2,3,4]
Output: [1,3,6,10]
Explanation: Running sum is obtained as follows: [1, 1+2, 1+2+3, 1+2+3+4].
Example 2:
Input: nums = [1,1,1,1,1]
Output: [1,2,3,4,5]
Explanation: Running sum is obtained as follows: [1, 1+1, 1+1+1, 1+1+1+1, 1+1+1+1+1].
Example 3:
Input: nums = [3,1,2,10,1]
Output: [3,4,6,16,17]
Constraints:
1 <= nums.length <= 1000
-10^6 <= nums[i] <= 10^6
UNLOCK POTENTIAL
New report reveals IT leaders' views
HPE
8.9K
218
134 Online
Code
Java
Auto
class Solution {
public int[] runningSum(int[] nums) {
int sum = 0;
for (int i = 0; i < nums.length; i++) {
sum = sum + nums[i];
nums[i] = sum;
}
return nums;
}
}
Saved
Ln 13, Col 2
Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
nums =
[1,2,3,4]
Output
[1,3,6,10]
Expected
[1,3,6,10]
Contribute a testcase
34°C
Mostly cloudy
Search
ENG
IN
11:24
28-07-2026


Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
1672. Richest Customer Wealth
Easy
Topics
Companies
Hint
You are given an m x n integer grid accounts where accounts[i][j] is the amount of money the i th customer has in the j th bank. Return the wealth that the richest customer has.
A customer's wealth is the amount of money they have in all their bank accounts. The richest customer is the customer that has the maximum wealth.
Example 1:
Input: accounts = [[1,2,3],[3,2,1]]
Output: 6
Explanation:
1st customer has wealth = 1 + 2 + 3 = 6
2nd customer has wealth = 3 + 2 + 1 = 6
Both customers are considered the richest with a wealth of 6 each, so return 6.
Example 2:
Input: accounts = [[1,5],[7,3],[3,5]]
Output: 10
Explanation:
1st customer has wealth = 6
2nd customer has wealth = 10
3rd customer has wealth = 8
The 2nd customer is the richest with a wealth of 10.
Example 3:
Input: accounts = [[2,8,7],[7,1,3],[1,9,5]]
Output: 17
Constraints:
m == accounts.length
4.9K
108
39 Online
Code
Java
Auto
class Solution {
public int maximumWealth(int[][] accounts) {
int maxWealth = 0;
for (int i = 0; i < accounts.length; i++) {
int sum = 0;
for (int j = 0; j < accounts[i].length; j++) {
sum += accounts[i][j];
}
if (sum > maxWealth) {
maxWealth = sum;
}
Saved
Ln 4, Col 1
Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
accounts =
[[1,2,3],[3,2,1]]
Output
6
Expected
6
Contribute a testcase
34°C
Mostly cloudy
Search
ENG
IN
11:32
28-07-2026

Description | Editorial | Solutions | Submissions

Given an integer array `nums` sorted in **non-decreasing** order, return *an array of **the squares of each number** sorted in non-decreasing order.*

Example 1:

```
Input: nums = [-4,-1,0,3,10]
Output: [0,1,9,16,100]
Explanation: After squaring, the array becomes [16,1,0,9,100].
After sorting, it becomes [0,1,9,16,100].
```

Example 2:

```
Input: nums = [-7,-3,2,3,11]
Output: [4,9,9,49,121]
```

Constraints:

- `1 <= nums.length <= 10^4`
- `-10^4 <= nums[i] <= 10^4`
- `nums` is sorted in **non-decreasing** order.

10.4K | 194 | 146 Online

Code

Java | Auto

```java
class Solution {
    public int[] sortedSquares(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];
        int left = 0;
        int right = n - 1;
        int index = n - 1;
        while (left <= right) {
```

Saved | Ln 19, Col 6

Testcase | Test Result

nums =

[-4,-1,0,3,10]

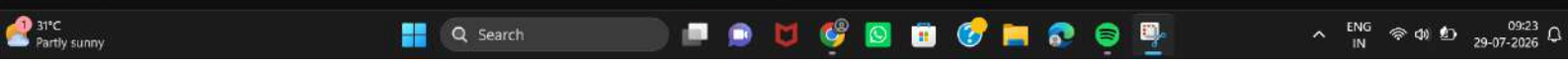
Output

[0,1,9,16,100]

Expected

[0,1,9,16,100]

Contribute a testcase



Description | Editorial | Solutions | Submissions

724. Find Pivot Index

Easy | Topics | Companies | Hint

Given an array of integers `nums`, calculate the **pivot index** of this array.

The **pivot index** is the index where the sum of all the numbers **strictly** to the left of the index is equal to the sum of all the numbers **strictly** to the index's right.

If the index is on the left edge of the array, then the left sum is `0` because there are no elements to the left. This also applies to the right edge of the array.

Return *the **leftmost pivot index***. If no such index exists, return `-1`.

Example 1:

```
Input: nums = [1,7,3,6,5,6]
Output: 3
Explanation:
The pivot index is 3.
Left sum = nums[0] + nums[1] + nums[2] = 1 + 7 + 3 = 11
Right sum = nums[4] + nums[5] = 5 + 6 = 11
```

Example 2:

9.5K | 273 | 135 Online

Code

Java | Auto

```java
class Solution {
    public int pivotIndex(int[] nums) {
        int totalSum = 0;
        for (int num : nums) {
            totalSum += num;
        }
        int leftSum = 0;
        for (int i = 0; i < nums.length; i++) {
            int rightSum = totalSum - leftSum - nums[i];
```

Saved | Ln 17, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

nums =
[1,7,3,6,5,6]

Output

3